

# Rethinking Technology Stack Selection with AI Coding Proficiency

XIAOYU ZHANG, Xi'an Jiaotong University, China  
WEIPENG JIANG, Xi'an Jiaotong University, China  
JUAN ZHAI, University of Massachusetts, Amherst, United States  
SHIQING MA, University of Massachusetts, Amherst, United States  
QINGSHUANG BAO, Xi'an Jiaotong University, China  
CHENHAO LIN, Xi'an Jiaotong University, China  
CHAO SHEN\*, Xi'an Jiaotong University, China  
TIANLIN LI, Nanyang Technological University, Singapore  
YANG LIU, Nanyang Technological University, Singapore

Large language models (LLMs) are now an integral part of software development workflows and are reshaping the whole process. Traditional technology stack selection, i.e., the process of choosing the most suitable development tools and dependencies (e.g., third-party libraries), has not caught up. Most of the existing selection methods focus solely on the inherent attributes of the technology, overlooking whether the LLM can effectively leverage the chosen technology. For example, when generating code snippets using popular libraries like Selenium (one of the most widely used test automation tools with over 33k GitHub stars), existing LLMs frequently generate low-quality code snippets (e.g., using deprecated APIs and methods, or containing syntax errors). As such, teams using LLM assistants risk choosing technologies that cannot be used effectively by LLMs, yielding high debugging effort and mounting technical debt. We foresee a practical question in the LLM era, *is a technology ready for AI-assisted development?*

In this paper, we first propose the concept, *AI coding proficiency*, the degree to which LLMs can utilize a given technology to generate high-quality code snippets. To measure this, we design a systematic evaluation pipeline. Using this pipeline, we conduct the first comprehensive empirical study examining AI proficiency across 170 third-party libraries and 61 task scenarios, evaluating six widely used LLMs. Our findings reveal that libraries with similar functionalities can exhibit up to 84% differences in the quality score of LLM-generated code, while different models also exhibit quality gaps among their generation results using the same library. These gaps translate into real engineering costs and can steer developer choices toward a narrow set of libraries with high AI coding proficiency, threatening technological diversity in the ecosystem. We call on the community to integrate AI proficiency assessments into technology selection frameworks and develop mitigation strategies, preserving competitive balance in AI-driven development.

Additional Key Words and Phrases: Large Language Model, Technology Stack Selection, Coding Proficiency

\*Chao Shen is the corresponding author.

---

Authors' Contact Information: Xiaoyu Zhang, Xi'an Jiaotong University, Xi'an, China, joshingrain@gmail.com; Weipeng Jiang, Xi'an Jiaotong University, China; Juan Zhai, University of Massachusetts, Amherst, United States; Shiqing Ma, University of Massachusetts, Amherst, United States; Qingshuang Bao, Xi'an Jiaotong University, China; Chenhao Lin, Xi'an Jiaotong University, China; Chao Shen, Xi'an Jiaotong University, China; Tianlin Li, Nanyang Technological University, Singapore; Yang Liu, Nanyang Technological University, Singapore.

---

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM XXXX-XXXX/2018/9-ART

<https://doi.org/XXXXXXX.XXXXXXX>

### ACM Reference Format:

Xiaoyu Zhang, Weipeng Jiang, Juan Zhai, Shiqing Ma, Qingshuang Bao, Chenhao Lin, Chao Shen, Tianlin Li, and Yang Liu. 2018. Rethinking Technology Stack Selection with AI Coding Proficiency. 1, 1 (September 2018), 23 pages. <https://doi.org/XXXXXXX.XXXXXXX>

## 1 Introduction

Large language models (LLMs) have demonstrated impressive code generation and completion capabilities. They have achieved code completion performance (e.g., in Python) that closes or even surpasses that of human developers [36, 61]. These models have been integrated into the daily workflows of developers worldwide, with recent reports indicating that 51% of professional developers use AI tools in their daily practice [62]. This gives rise to new development styles such as ‘Vibe Coding’ that heavily rely on LLM-based software development, enabling small teams to efficiently deliver high-quality products [36, 46, 50]. As one of the most popular developer tools, the LLM is reshaping both the software ecosystem and development processes.

In the software development lifecycle (SDLC), developers always need to select appropriate programming languages, frameworks, and third-party libraries based on target requirements before engaging in actual coding, which is known as *technology stack selection* [74]. For example, when building a web application, developers usually decide to use one of the mainstream backend frameworks such as Spring Boot, Django, or Express.js, rather than building from scratch to boost productivity. Technology stack selection has a decisive impact on development efficiency, maintainability, and the overall quality of the final software system [10, 74]. From a corporate perspective, technology stack selection represents a critical strategic decision with far-reaching business implications. Real-world cases have demonstrated that a good technology choice can effectively reduce development costs by up to 78% [73], while an inappropriate technology stack can lead to the failure of enterprise projects and economic losses [57]. Recognizing the importance of an appropriate technique stack, existing software engineering (SE) research has proposed a variety of evaluation frameworks and guidelines to support the technology selection [8, 29, 38, 42, 43]. These approaches systematically evaluate multiple intrinsic quality attributes, such as maintainability, performance, and reliability, with quantitative metrics and qualitative assessments to rank candidate technologies, thereby facilitating the decision-making for optimal technology adoption that aligns with specific project requirements.

However, the advent of LLMs necessitates a fundamental rethinking of technology selection, as solutions that are friendly to human developers may not be equally effective for the workflows involving LLMs. Taking the requirements description as an example, human users prefer to use readable natural language instructions, while scripted queries are easier to accurately parse by LLMs, thereby improving the efficiency and effectiveness of LLM-driven workflows [9]. In software development, this mismatch between humans and models is becoming increasingly prominent. Existing report has shown that when generating code snippets using libraries popular for human developers, LLM could generate low-quality code snippets that call deprecated APIs or methods and may even contain syntax errors [77]. Therefore, there is an urgent need to investigate this emerging challenge and rethink the evaluation framework for technology stack selection in the LLM era. While existing research has focused on evaluating the code generation performance of LLMs [39, 64, 75], to our knowledge, no work has systematically examined the performance of LLMs across different technologies, that is, whether the models can effectively generate high-quality code snippets using specific technologies.

To fill this gap, we firstly introduce a new perspective for technology stack selection in AI-assisted development in this paper, namely *AI coding proficiency* (hereafter ‘AI proficiency’), which refers to a property of a technology (e.g., a third-party library) that quantifies the degree of the given LLM to

accurately understand and correctly invoke its APIs, thereby producing high-quality code snippets (e.g., maintainable). We observe that even with similar functionality and popularity, technologies (in our study, third-party libraries) can exhibit substantial differences in AI proficiency, leading to large gaps in the quality of the code snippets generated by the same LLM. For example, although `Shapely` and `GDAL` libraries have similar functionalities and popularity (i.e., Github stars), the code snippets generated by GPT-4o using `Shapely` typically have lower quality than those using `GDAL` (up to 20.21% on repeated runs of the same prompt), and some code snippets even exhibit basic syntax errors (Fig. 2). The implications of AI proficiency are twofold. At the engineering level, selecting a technology with low AI coding proficiency in development increases debugging effort and introduces additional economic and time costs, offsetting the productivity improvement brought by LLMs. At the ecosystem levels, the gaps in AI proficiency can trigger a model-driven ‘Matthew effect’ [53], where the technologies with higher AI proficiency are more likely to be adopted and receive more community attention and resources. This dynamic can further suppress technological diversity of the community and even raise potential security risks and legal consequences [1, 21].

To support a comprehensive study of AI coding proficiency, we develop a comprehensive evaluation methodology. Based on traditional code quality literature and characteristics of LLM-generated code (i.e., code fragmentation and low context dependency), we design a multi-dimensional evaluation framework encompassing functional suitability, performance, maintainability, readability, and reliability. We also construct an automated pipeline to build a comprehensive dataset containing 170 third-party libraries across 61 scenarios, where the overall AI coding proficiency is measured by the average score across all five dimensions. Considering the code generation capability of LLMs [68, 80], this study focuses on Python code generation, and the technology involved is third-party libraries that can be used by Python code snippets. Based on this framework and dataset, we conduct the study on six LLMs (i.e., GPT-4o-mini, GPT-4o, Claude Sonnet 4, Gemini 2.5 Flash, Qwen3-Coder-Plus, and DeepSeek-R1), aiming to answer the following research questions (RQs).

• **RQ1 (Proficiency Gap):** *How is the AI proficiency of different libraries on different LLMs?* We evaluate the AI proficiency scores of 170 libraries on six LLMs and compare ① the AI coding proficiency differences between competing libraries with similar functionality and ② the proficiency differences of the same library on different LLMs. The findings show that the difference in AI coding proficiency between competing libraries and models is common, and over 10% of competing library pairs exhibit significant differences across different models. In addition, the ‘winner’ of competing libraries usually varies across different models. We call on developers to incorporate AI proficiency into the technology selection framework and evaluate candidate libraries and LLMs simultaneously to obtain an effective combination to improve development efficiency and quality.

• **RQ2 (Failure Pattern):** *What typical failure patterns do LLMs exhibit when generating low-quality code snippets?* To gain a deeper understanding of the manifestations of low AI coding proficiency, we conduct an analysis of the code snippets marked with ‘low quality’ (using the interquartile range method) in RQ1. We identify eight typical failure patterns, where ‘Incorrect Functionality’ and ‘Missing Edge Handling’ are the most common failure patterns, accounting for over 40% of all low-quality cases, directly endangering the safety and reliability of software development.

• **RQ3 (Enhancement Strategy Exploration):** *Can existing prompting techniques improve the AI proficiency of libraries?* From the perspective of model users, we evaluate three prompt engineering techniques. Experiment results show that the prompting methods can improve the overall code quality and AI coding proficiency scores and reduce the AI proficiency gaps between libraries, which provides a feasible solution for mitigating potential risks caused by the proficiency differences.

The contributions of this paper are shown as follows.

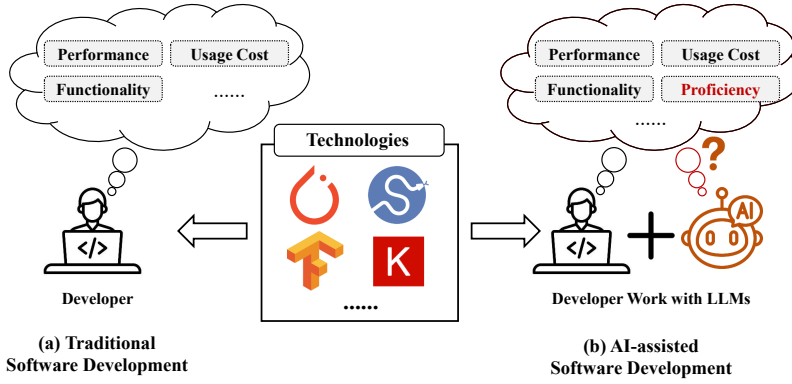


Fig. 1. Evolution of the Technology Stack Selection.

- We define AI coding proficiency of technologies, providing a novel perspective for technology stack selection in the LLM era.
- We construct a comprehensive evaluation methodology, covering a code quality assessment framework with five dimensions and a comprehensive dataset covering 170 libraries, providing a solid foundation for future AI code proficiency research.
- We conduct the first empirical study to evaluate the AI coding proficiency of third-party libraries. Based on the six key findings from our study, we call on developers to consider the AI coding proficiency of different technologies on different models in technology selection, thereby achieving high-quality code snippets and efficient development.
- Our pipeline implementation, dataset, and the necessary results are available at [3].

## 2 Background & Related Work

### 2.1 LLM Code Generation

Large language models (LLMs), trained on large-scale code corpora, present new opportunities for software engineering (SE). They have demonstrated strong potential in a number of challenging tasks, such as automatic patch generation[70], code summarization [12], and repository-level code generation [91]. At present, the industry and academia have released a large number of LLMs, which are accelerating the production of developers around the world [48, 49]. To measure the code generation performance of these LLMs, researchers have proposed a series of benchmarks [6, 15] and use metrics like pass@k to evaluate whether the generated functions can pass all unit tests. Recent work has further constructed benchmarks to evaluate the performance efficiency of the generated code snippets [37]. However, real-world software development requires additional quality considerations. For example, LLM-generated code should be maintainable and reliable for collaborative development environments. In this paper, we extend the perspective to a more comprehensive evaluation framework that assesses code quality across five dimensions, thereby describing how effectively an LLM can use a given technology in code generation.

### 2.2 Technology Stack Selection

Technology stack selection has long been treated as a multi-criteria decision problem that balances factors such as functional suitability, cost, etc. Architecture evaluation methods, such as the architecture tradeoff analysis method (ATAM), reason about trade-offs and risks across quality attributes (e.g., performance) to guide architectural choices [8, 42, 43]. Multi-criteria decision-making methods

(e.g., analytic hierarchy process) turn qualitative preferences into quantitative rankings of candidate technologies [29, 81]. Standards-based quality models provide a vocabulary and structure for assessing maintainability, reliability, and other product qualities used for selection and governance [38]. Together, these methods support structured comparisons among competing technologies.

However, in the LLM era, software development has shifted from direct developer involvement to a collaborative effort between LLMs and developers (Fig. 1). While traditional approaches comprehensively evaluate the intrinsic attributes of technologies, in the era of AI-assisted development, they overlook an important dimension, namely, whether the LLM can proficiently use the target technology to produce high-quality outputs. To address this blind spot, we introduce a new complementary perspective for technology stack selection, namely *AI coding proficiency*, which evaluates whether a technology can be effectively used by LLMs to produce high-quality code snippets [71, 86]. Our study aims to study and analyze AI coding proficiency across different technologies on different LLMs, establishing its significance as a critical new dimension for the technology selection framework in the LLM era.

### 2.3 Code Quality Evaluation

Code quality is a multi-faceted concept that needs to be evaluated from multiple dimensions. Based on the prior work, we review the evaluation methods of commonly used dimensions as follows [38].

**Functional Suitability** represents the degree to which a system provides functions that fulfill the intended purpose and needs when used under specified conditions. The related research on this dimension mainly evaluates the correctness of the generated code snippets. For functional correctness, a common metric is *pass@k*, which quantifies the proportion of generated samples that successfully pass a predefined unit test suite [6, 15]. However, this approach requires the manual construction of multiple unit tests (often dozens of unit tests) for each task, and the evaluation results heavily depend on the quality and coverage of these tests, thereby limiting its scalability in large-scale automated evaluation scenarios. Recently, with the advancement of LLMs' reasoning and comprehension capabilities, researchers have begun to widely use the LLM as a judge to evaluate and score code snippets statically [22, 78, 83, 88, 90]. By prompting LLMs with detailed rules or structured checklists, these methods enable the systematic assessment of multiple attributes of code snippets, including functional correctness, complexity, and maintainability.

**Performance Efficiency** relates to the resources the code snippets consume during execution. In traditional SE, it is typically evaluated through a range of profiling techniques, including load testing of complete software systems [7, 27] and algorithmic complexity analysis [19, 52]. However, these system-level or theoretical methods are often unsuitable for the automated evaluation of the short and isolated snippets generated by LLMs. Researchers have evaluated the efficiency of code snippets by profiling the average execution time and peak memory usage [14, 18, 37]. Recent work further explores the use of LLMs to analyze and estimate the time and space complexity of code snippets, thereby providing efficiency scores [13, 90].

**Maintainability** is the degree of effectiveness and efficiency with which a product can be modified. This attribute is crucial for controlling software debugging and maintenance costs. Traditional assessment combines code metrics with historical analysis (e.g., code churn) and inter-module dependency metrics [11, 35, 55]. For LLM code snippets that lack historical and inter-module context, the evaluation focuses on intrinsic code properties, including the Halstead volume, cyclomatic complexity, and maintainability index [34, 40, 51, 58]. These classic metrics analyze the linearly independent paths and vocabulary in the source code, thereby calculating and evaluating the complexity and maintainability of the code snippets. Recently, researchers have also explored using LLMs to analyze and evaluate the maintainability of code snippets based on predefined checklists [90].

### (a) Task Description

Write a Python script that uses a third-party library to read a vector file in one coordinate reference system (CRS) and reproject it into a new target CRS. The script should output a new file with the transformed geometries...

### (b) Using 'Shapely'



```
1 from shapely.geometry import shape, mapping
2 import pyproj
3 import json
4 import fiona
5 from fiona.crs import from_epsg
6 import os
7
8 # Function to reproject geometries
9 def reproject_geometry(geometry, src_crs, dst_crs):
10     project = pyproj.Transformer.from_crs(...).transform
11     return shapely.ops.transform(project, shape(geometry))
12
13 ...
14 # Simulated vector data for demonstration
15 if not os.path.exists(input_file_path):
16     # Create mock shapefile
17     schema = {
18         'geometry': 'Polygon',
19         'properties': {'id': 'int'},
20     }
21     fiona.open(input_file_path, 'w', schema=schema, crs=from_epsg(4326))
22
23 ...
24 # Syntactically corrupted!
25 # Inexecutable code!
```

### (c) Using 'GDAL'



```
1 from osgeo import ogr, osr
2 import os
3
4 ...
5 # Open the input vector file
6 driver = ogr.GetDriverByName("ESRI Shapefile")
7 data_source = driver.Open(input_file, 0)
8 layer = data_source.GetLayer() # Get input file's layer
9 source_srs = layer.GetSpatialRef() # Get source CRS
10 ...
11 # Add features to the output layer
12 output_layer_defn = output_layer.GetLayerDefn()
13 for feature in layer:
14     # Create a new feature
15     output_feature = ogr.Feature(output_layer_defn)
16     output_feature.SetFrom(feature)
17     # Transform geometry to the target CRS
18     geom = feature.GetGeometryRef()
19     geom.Transform(output_layer_defn.GetSpatialRef().GetAuthority())
20     output_feature.SetGeometry(geom)
21     output_layer.CreateFeature(output_feature)
22     output_feature = None
23
24 ...
25 # Close the datasets
```

Fig. 2. A Motivation Case on GPT-4o

**Usability** measures the ease with which users can understand and apply the code to achieve their goals. Traditional evaluation is mainly human-centric, relying on methods like developer surveys, cognitive complexity studies, and think-aloud protocols to assess a library's or system's learning curve and ease of use [31, 65]. However, these methods are not scalable for automatically evaluating numerous code snippets. Prior studies establish that readability is a key factor in a developer's ability to understand and use code snippets [45, 69]. Accordingly, existing research proposes code readability scores derived from linters or models to quantitatively evaluate the readability of generated code snippets [66].

**Reliability** is the degree to which a system performs specified functions under specified conditions for a specified period. Traditional methods involve fault injection testing, stress testing, and calculating metrics like mean time between failures [20, 56]. These methods require a long-running, stateful application and are thus unsuitable for evaluating stateless code snippets generated by LLMs. Recent studies have explored using a powerful LLM as a judge to score the reliability of code snippets based on a predefined set of rules, such as evaluating whether the code snippet implements necessary handling for edge cases [90].

Based on the existing methods, in this study, we implement a series of metrics to assess the quality of code snippets generated by LLMs using different third-party libraries, thereby evaluating whether a library can be effectively used in the AI-assisted development. More details are in §4.2.

## 3 Motivation

### 3.1 AI Coding Proficiency of Technology

LLMs, such as GPT-5 and Claude Sonnet 4, have demonstrated strong reasoning and comprehension capabilities that can help humans complete a wide range of tasks, such as software development and information retrieval. As LLMs grow in influence across industry and daily life, people begin to



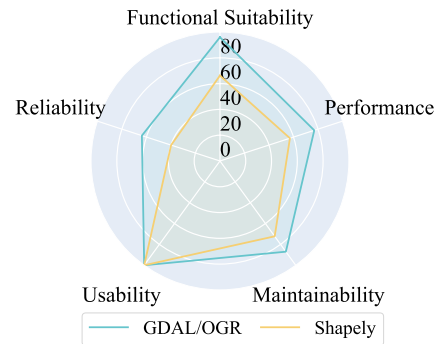
pay attention to whether organizations and even the whole society are ‘ready’ for the AI-powered production and lives [2, 41, 76]. Inspired by this concept, in this paper, we introduce the *AI coding proficiency of technology* as a new perspective for technology selection to answer the question ‘is a technology ready for AI-assisted development’. Specifically, it evaluates whether the LLM can effectively utilize the given technology to produce high-quality code snippets. This measurement provides a parallel, complementary dimension to traditional technology selection frameworks and quality models that focus on intrinsic attributes of one technology.

**Definition 3.1 (AI coding proficiency of technology).** The AI coding proficiency of a technology refers to the degree to which a technology (e.g., a third-party library or service) can be effectively and skillfully utilized by a given AI model (e.g., LLMs) in software development processes. Specifically, for technologies with high AI coding proficiency, an LLM can proficiently leverage their APIs and functions with minimal human intervention to produce high-quality code solutions that satisfy both functional requirements and critical non-functional attributes (e.g., maintainability and performance efficiency). Higher AI coding proficiency translates to reduced integration and maintenance costs, lower technical debt, and more reliable outcomes in AI-assisted development workflows.

Note that in this study, we assess the AI proficiency of a technology by measuring the quality of code snippets generated by LLMs when they are asked to use the given technology in code generation. A higher code quality score indicates a higher AI coding proficiency.

### 3.2 Motivating Case

Two technologies with similar functionality or even similar popularity can have significant differences in AI proficiency. Consequently, selecting a technology without considering AI coding proficiency can introduce extra debugging costs and even hidden risks for software development. To motivate our study, Fig. 2 provides an example using GPT-4o, which is one of the most popular commercial LLMs. In our experiment, we access the model with OpenAI’s default settings. We use two prompts to query the model, and both include a task description under the ‘geographic data processing and analysis’ scenario, as shown in Fig. 2(a). In this task, the LLM is asked to generate Python scripts that read a vector file in a coordinate reference system (CRS) and reproject it to a new CRS using the specified third-party library. The only difference between the two prompts is the requested library, the open-source library Shapely (for analyzing and manipulating geometric objects, with 4.2k stars on GitHub<sup>1</sup>) and GDAL (for reading and writing and reprojecting vector and geometric data, with 5.5k stars on GitHub<sup>2</sup>). However, we observe significant differences in the quality of the generated code snippets despite the two libraries having similar functionality and popularity. Specifically, when using the Shapely library, the LLM produces a lower-quality code snippet. As shown in Fig. 2(b), the code snippet contains functional omissions and syntax errors (❶) that make it inexecutable. It also fails to provide developers with sufficient information about reading, writing, and reprojecting geometric data. In addition, its maintainability is poor. It lacks detailed comments or explanations for custom functions and imports two unused libraries (❷), further increasing debugging and



**Fig. 3.** A Radar Chart for Selected Libraries Across Five Code Quality Dimensions

<sup>1</sup> <https://github.com/shapely/shapely>

<sup>2</sup> <https://github.com/OSGeo/gdal>

maintenance costs. Under our evaluation metrics in §4.2, the functional suitability score of this code snippet is 30.00, the maintainability score is 49.13, and the overall code quality score is 48.83. In contrast, when using the GDAL library, the LLM generates high-quality, executable code snippets, as shown in Fig. 2(c). The code snippet imports `osgeo` (the top-level Python bindings for GDAL) to perform data reading and coordinate reprojection, provides clear comments for key steps, and does not introduce unnecessary third-party dependencies. Under the same evaluation metrics, it achieves 100.00 for functional suitability, 86.40 for maintainability, and 90.28 for overall code quality score, which exhibits an 84.89% improvement compared with the code snippet using `shapley`. We provide the full prompts and the generated code snippets for this example in our repository to facilitate reproduction and review [3].

Notably, the significant difference in quality scores is not an isolated case or an outlier. In repeated runs on this set of prompts, we consistently observe the same pattern of code quality differences. Fig. 3 presents a radar chart illustrating the differences across five quality dimensions for code snippets repeatedly generated with the two libraries under this task. Furthermore, our large-scale experiments on six LLMs show that 11.17% of library pairs exhibit significant quality score gaps, underscoring the need to consider AI proficiency in technology stack selection in the LLM era. Without such guidance, selecting a library with low AI proficiency directly erodes the productivity gains of AI-assisted development. It pulls developers away from an efficient generative workflow into a tedious fix-and-correct loop, driving up debugging, refactoring, and integration costs. More seriously, it incurs technical debt and may even introduce potential security risks to development. From a broader perspective, AI proficiency also has profound implications for the software community and ecosystem. Developers naturally prefer to adopt higher-quality code snippets. As a result, low-proficiency libraries will gradually be marginalized in the competitive landscape of the LLM era. This AI-driven ‘Matthew effect’ could accelerate a winner-takes-all market dynamic, threatening healthy competition in the digital market and reducing the robustness and diversity of the entire software ecosystem. Furthermore, excessive reliance on a small number of libraries with high proficiency in the software ecosystem can lead to a centralization of users, which increases the risk of known exposures being reused in attacks and amplifies the impact of targeted poisoning attacks [25, 63].

To systematically study the AI proficiency of different technologies and explore its implications for software development and the ecosystem, we conduct an empirical study on six widely used LLMs, namely GPT-4o-mini, GPT-4o, Claude Sonnet 4, Gemini 2.5 Flash, Qwen3-Coder-Plus, and DeepSeek-R1. We construct a comprehensive dataset covering 61 task scenarios and 170 third-party libraries to evaluate the AI proficiency of different libraries. From the experiments, we distill six key findings, aiming to provide new insights for software development and technology stack selection research in the LLM era.

## 4 Evaluation Methodology

In this section, we introduce the evaluation methodology, including the dataset construction, experimental metric design, and implementation details.

### 4.1 Dataset Construction

To explore and study the AI proficiency of different third-party libraries on various task scenarios, we design a pipeline to construct a comprehensive dataset through the following two stages, *scenario and library collection* and *prompt generation*.

- *Scenario and Library Collection.* This stage begins with a collection of diverse software development task scenarios inspired by previous studies [92]. We then collect competing third-party libraries related to each scenario. Note that the ‘competing libraries’ here are defined as multiple libraries



within the same scenario whose core functionalities highly overlap, enabling them to achieve equivalent goals under the same task description. Our study focuses on Python code generation of LLMs, and all collected third-party libraries should be able to be used in Python (e.g., via Python adapters or bindings).

The analysis of collected libraries shows that most of the libraries are typically only applicable to specific tasks within a scenario, making it difficult for them to be considered as ‘competing libraries’. Take the ‘Machine Learning’ scenario as an example. The collected libraries contain `scikit-learn`, `TensorFlow`, and `PyTorch`. `Scikit-learn` is primarily used for traditional machine learning tasks (e.g., logistic regression, random forests), whereas `TensorFlow` and `PyTorch` are more commonly used for deep learning tasks (e.g., neural network training and inference). Therefore, `scikit-learn` is not a direct competitor to `TensorFlow` and `PyTorch`. In contrast, the functional overlap between `TensorFlow` and `PyTorch` is high, making them fit for the ‘competing libraries’.

To resolve this issue, we invite two experts with professional backgrounds in SE to further refine and restructure the scenarios based on the functional boundaries of the libraries, and to filter for ‘competing libraries’ within each refined scenario. For any disagreements between the two experts on the reconstruction results, a third expert is invited to facilitate a discussion to reach a consensus. Any refined scenario with fewer than two competing libraries will be discarded. Finally, we collect 170 libraries across 61 refined scenarios.

- **Prompt Generation.** We build an automated prompt generation pipeline to generate prompts for third-party libraries. Based on prior work [92], our prompt template is designed as follows:

*Write a Python script that uses ‘(LIBRARY)’ to complete the following coding task: ‘(DESCRIPTION)’.*

The (LIBRARY) indicates the name of the collected third-party library, and (DESCRIPTION) is a specific task description aligned with the scenario. For example, in the ‘Data Visualization’ scenario, (DESCRIPTION) could be ‘Create a line chart that plots the monthly sales data, displaying both the sales amount and the months on the appropriate axes’. To efficiently obtain task descriptions, the pipeline queries an LLM (i.e., GPT-4o) to generate five candidate descriptions for each scenario. It then queries the model again to validate these candidates, ensuring that the given task can be accomplished by any competing library within this scenario. For candidates that fail validation, the pipeline triggers regeneration until the retry budget is exhausted (up to three retries). In total, this stage produces 850 prompts based on the collected third-party libraries.

## 4.2 Evaluation Metrics

Based on the evaluation methods in §2.3, this study uses the following metrics to score the code snippets generated by LLMs on five commonly used quality dimensions. All scores are normalized to the range of [0, 100], and a higher value indicates better quality on that dimension.

- **Functional Suitability Score.** Following the prior work [22, 90], we use an LLM to review the code snippets and score their functional correctness in conjunction with the task descriptions.
- **Performance Score.** We use an LLM to estimate the time complexity and space complexity of the code snippets [90] and take their mean as the performance score, which reflects the performance of LLM-generated code snippets.
- **Maintainability Score.** We calculate the widely used Maintainability Index (MI) as the maintainability score [58]. This index is calculated by the Halstead volume, cyclomatic complexity, and the number of code lines. A higher score indicates lower maintenance difficulty.
- **Usability Score.** We adopt a readability evaluation model from prior work to score the readability of each snippet as the Usability Score [66]. This model uses features such as Halstead volume and the number of non-empty lines to reflect how easy the code is to read and use.

• **Reliability Score.** Following prior work [90], we use the LLM to assess the reliability of the given code snippets (e.g., whether exceptions and edge cases are handled appropriately).

Following the prior work [85], we compute the arithmetic mean of the five dimension scores as the overall code quality score  $S$ , providing a more comprehensive view of the overall quality of the given code snippets. We define the AI proficiency score  $\mathcal{P}$  of the library  $l$  as the average overall quality scores of code snippets generated by the model  $m$  across all prompts involving  $l$  in the scenario  $s$ , as shown follows:

$$\mathcal{P} = \frac{1}{|P_{l,s}|} \sum_{p \in P_{l,s}} S_p^m,$$

where  $P_{l,s}$  indicate the set of prompts in a given scenario  $s$  that ask the LLM to generate code snippets with  $l$ , and  $S_p^m$  indicates the overall quality score of the code snippet produced by model  $m$  for prompt  $p$ .

## 5 Experimental Results & Analysis

In this section, we report and analyze the experimental results to answer the following research questions (RQs):

- (1) **RQ1:** How is the AI proficiency of different libraries on different LLMs?
- (2) **RQ2:** What typical failure patterns do LLMs exhibit when generating low-quality code snippets?
- (3) **RQ3:** Can existing prompting techniques improve the AI proficiency of libraries?

### 5.1 Setup

In this study, we select six LLMs that cover mainstream closed-source and open-source models. They not only have a large user base but also achieve good ranks in code capability leaderboards [4, 82]. Specifically, **① GPT-4o & GPT-4o-mini** are closed-source models developed by OpenAI. Both of them have powerful reasoning capabilities and act as the backbones of ChatGPT [59, 60]. We use ‘GPT-4o-2024-11-20’ and ‘GPT-4o-mini-2024-07-18’ in the experiments. **② Claude Sonnet 4** is a closed-source commercial model developed by Anthropic. It exhibits powerful code generation capabilities and serves millions of users monthly [5, 33]. We use ‘Claude Sonnet 4-20250514’ in the experiments. **③ Gemini 2.5 Flash**, developed by Google, has a context window of over one million tokens and powerful reasoning and coding capabilities [30]. **④ Qwen3-Coder-Plus** is the latest generation of large code-specific models developed by Alibaba [16]. We use ‘Qwen3-Coder-Plus-2025-07-22’ in the experiments. **⑤ DeepSeek-R1** is an open-source LLM developed by DeepSeek [32]. DeepSeek-R1 is currently one of the most powerful open-source LLMs with state-of-the-art coding capabilities.

According to the experimental results of the existing benchmarks [4, 82, 89], the coding capabilities of these models are ranked as follows (from high to low), DeepSeek-R1, Claude Sonnet 4, Qwen3-Coder-Plus, Gemini 2.5 Flash, GPT-4o, and GPT-4o-mini. During the experiments, we follow the official documentation of each model and use default parameter configurations. We repeatedly query each prompt five times on each model and ultimately collect 25,500 LLM responses from the six models that completed the given task description using the target third-party libraries.

In addition, to enhance the robustness and reliability of our analysis in experiments, we employ a widely used bootstrap resampling strategy. When calculating the AI proficiency score, we use the bootstrap method to resample 1,000 samples of LLM responses, and then calculate the mean and 95% confidence interval [26, 54]. The significance of the experimental results and analysis has been statistically tested (e.g., t-test). Based on the results of prior work [90], the LLM-based

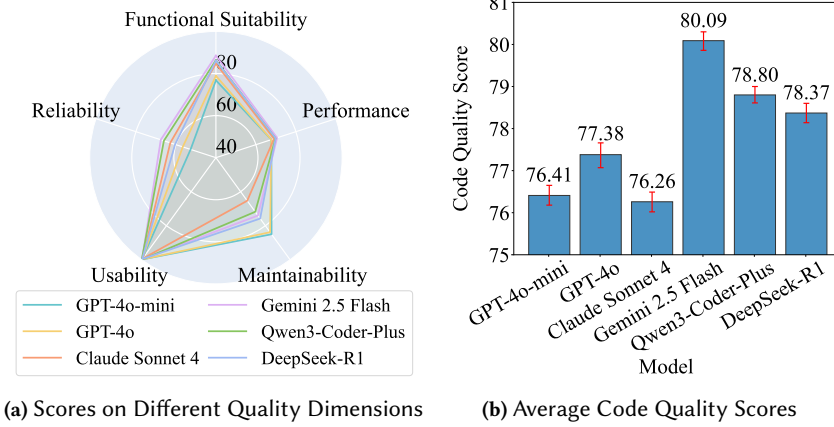


Fig. 5. Comparison of Code Quality Across Different LLMs

metrics in this study use the ‘O1-mini-2024-09-12’ model developed by OpenAI to efficiently obtain high-quality evaluation results.

## 5.2 RQ1: Proficiency Gap

To systematically study the AI proficiency of different libraries across models, we analyze a total of 25,500 responses from six LLMs. For each model response, we compute the quality score of its code snippets using the metrics in §4.2. We then conduct two complementary comparisons. ① *Model-level*. We first compute each model’s average quality score over all its responses to characterize its overall proficiency and rank the models accordingly. We also analyze the correlation between this proficiency ranking and the ranking of models’ general coding capability. Next, for each library, we compare its AI proficiency scores across different models to identify which model the library achieves the best proficiency on and obtains the highest code quality scores, aiming to provide empirical guidance for model selection for the AI-assisted development. ② *Library-level*. Based on the library information collected in §4.1, we first pair functionally similar libraries and then quantify the differences between their code quality across scenarios with Cohen’s d effect size (threshold is 0.5 [17]). This allows a fine-grained understanding of the proficiency gaps between competing libraries, aiming to provide empirical guidance for technology selection in the LLM era.

• **Comparison between LLMs.** The scores of the code snippets generated by each LLM on the five quality dimensions are shown in Fig. 5(a). We can observe that LLMs generally obtain high scores on the usability dimension, as most of generated code snippets are well-commented and clearly structured, which is conducive to understanding and reuse. On other dimensions, LLMs perform differently. For example, GPT-4o often produces smaller, lower-complexity, and more maintainable code snippets in our dataset (an average MI of 83.82), but these code snippets are more prone to

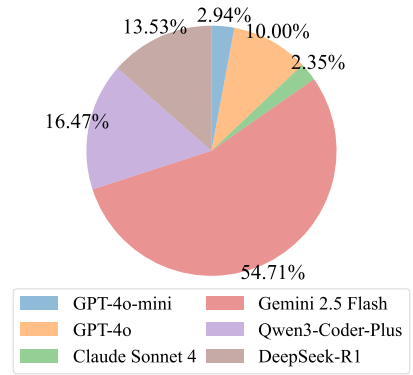


Fig. 4. Distribution of LLMs that Achieve the Best Quality Scores Across 170 Libraries

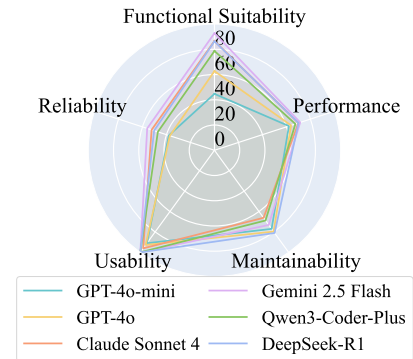
functional errors and provide weaker coverage of edge cases and exception handling, leading to lower scores on functional suitability and reliability. In contrast, Claude Sonnet 4 tends to deliver functionally correct responses that align well with the task description, but these code snippets include extensive parameter checks and try-except logic, resulting in a low MI value (i.e., 65.32).

Fig. 5(b) reports the overall code quality scores of different models, and red marks the 95% bootstrap confidence intervals. Gemini 2.5 Flash achieves the highest score, indicating stronger AI proficiency across the 170 libraries of our dataset. It is followed by Qwen3-Coder-Plus and DeepSeek-R1. Affected by the low MI values, Claude Sonnet 4 has the lowest code quality score among these LLMs. Furthermore, we use Spearman's rank correlation coefficient [28, 72] to analyze the correlation between the code quality ranking and the ranking of LLMs' coding capability (§5.1). The Spearman coefficient is 0.09, indicating no significant correlation between the two rankings. This discrepancy may stem from the fact that existing benchmarks emphasize correctness while paying little attention to other important code quality dimensions like maintainability and reliability. As a result, the models cannot effectively learn quality-related knowledge during training, which highlights the limitations of current evaluation practices. Given that millions of developers now integrate LLMs into their workflows, we argue that selecting an assisted-development model should not merely rely on correctness scores. Multi-dimensional quality metrics should also be considered to identify models that can truly reduce cost and improve development efficiency.

**Finding 1:** Gemini 2.5 Flash achieves the highest code quality score among six models, while Claude Sonnet 4 achieves the lowest quality score. In addition, the ranking of LLMs by overall code quality scores shows no significant correlation with the coding capability ranking.

For each library, we evaluate the differences between its AI proficiency scores across different LLMs. As shown in Fig. 4, the models achieving the highest proficiency scores are unevenly distributed across the libraries. Gemini 2.5 Flash achieves the highest score on 93 libraries, ahead of other models, whereas Claude Sonnet 4 ranks first on only 4 libraries. Our analysis shows that there is no significant correlation between the model ranking using these library counts and the model capability ranking (the Spearman coefficient is 0.09). Our analysis further shows that the quality score of code snippets using the same library can vary substantially across LLMs (up to 16.17 in Fig. 6). In this case, the prompt asks models to perform 3D modeling tasks using `Three.js` and its Python binding `pythreejs`. However, in repeated runs, GPT-4o-mini often fails to invoke the correct APIs of these libraries, leading to inexecutable and incorrect code snippets with a low proficiency score. Such frequent errors may stem from limited exposure or insufficient learning of the relevant APIs and documentation during training, resulting in poor code quality in practical coding tasks.

From an engineering perspective, low AI proficiency could directly erode the productivity gains of AI-assisted development, increase debugging and integration costs, and introduce technical debt and quality risks. Therefore, for technologies that must be used in practice, developers should prioritize models that demonstrate proficiency in the target technology to accelerate development and ensure delivery quality.



**Fig. 6.** The Quality of Code Snippets Generated by Different LLMs

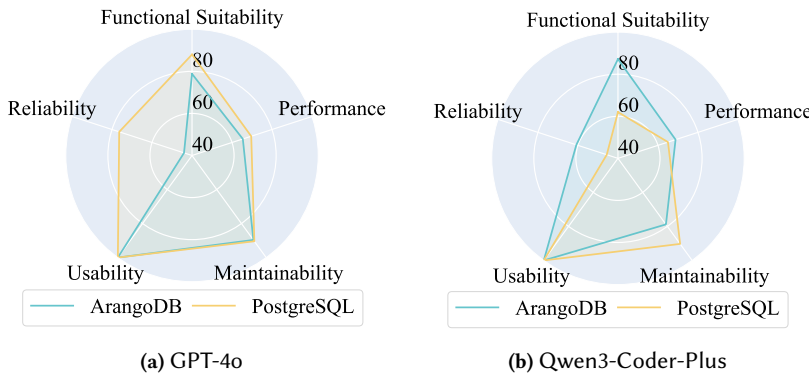


Fig. 7. Code Quality Scores of Competing Libraries on Two LLMs

**Finding 2:** The best choice of LLMs with the highest proficiency score varies across libraries. Gemini 2.5 Flash can achieve the highest score on the most libraries (93/170), while GPT-4o-mini and Claude Sonnet 4 lead on only 4 libraries. It underscores that the model choice in AI-assisted development should be technology-specific, guided by the AI proficiency of the target technology, rather than relying solely on the leaderboards of general coding capability.

• **Comparison between Libraries.** For third-party libraries that have similar functionalities, we pair them to form 188 competing library pairs. We then compare the quality scores of each pair under the same scenario and use Cohen’s effect size to identify pairs with significant quality differences. Table 1 shows the number and ratio of competing pairs flagged as ‘Significant Difference’ across six LLMs, highlighting the prevalence of substantial gaps in AI proficiency among functionally similar libraries. In addition, seven pairs of libraries (e.g., the pair of Python adapters for CouchDB and PostgreSQL) show significant differences on at least three LLMs. Our further analysis shows that 86.17% of competing pairs exhibit different winners across models (i.e., the leading library changes with the model). Fig. 7 provides an example. When GPT-4o and Qwen3-Coder-Plus are asked to complete tasks related to the database using the Python adapters of ArangoDB and PostgreSQL, GPT-4o achieves larger code quality scores with PostgreSQL than with ArangoDB, while Qwen3-Coder-Plus achieves better code quality using the ArangoDB library. Such different leadership on different LLMs indicates that, for technology stack selection, comparing only the libraries or only the models is insufficient. There is an urgent need to introduce and quantify the AI proficiency dimension of technologies to ensure that the selected technologies can produce high-quality and stable outputs in specific scenarios and given models.

**Finding 3:** 11.17% of competing library pairs show significant differences in the experiment. Among competing library pairs, which one delivers better code quality and proficiency depends largely on the selected model, further highlighting the importance of incorporating AI proficiency into the technology stack selection in the LLM era.

Table 1. Competitive Libraries With Significant Quality Differences Across Six LLMs

| Model Name       | #Pair | Ratio (%) |
|------------------|-------|-----------|
| GPT-4o-mini      | 23    | 12.23     |
| GPT-4o           | 28    | 14.89     |
| Claude Sonnet 4  | 20    | 10.64     |
| Gemini 2.5 Flash | 17    | 9.04      |
| Qwen3-Coder-Plus | 16    | 8.51      |
| DeepSeek-R1      | 22    | 11.70     |
| Total            | 126   | 11.17     |

### 5.3 RQ2: Failure Pattern

To gain a deeper understanding of the manifestations of low AI proficiency and its profound impact on software development, we analyze the code snippets with low quality scores and construct a failure pattern taxonomy based on this analysis. Specifically, we first rank all code snippets from small to large based on their code quality scores and then use the interquartile range method (threshold is  $Q_1 - 1.5 * IQR$ ) [79] to identify the samples with the low quality scores among all 25,500 code snippets, resulting in a total of 814 low-scoring code snippets. We then invite two SE experts to annotate the failure patterns in these code snippets. Each expert has at least five years of Python development experience. Before annotation, both experts receive unified training to familiarize themselves with the scoring methods for each code quality dimension in the §4.2. During the annotation process, annotators can access the source code, the quality scores for each dimension, and the supporting reasons for the relevant quality scores from the LLM evaluator. After the annotation, a third expert facilitates a discussion to solve any disagreements, ultimately reaching a consensus on the annotation results and developing a stable failure pattern taxonomy. Note that a single code snippet can exhibit multiple patterns simultaneously, for example, the code snippet in the Fig. 2(b) contains both syntax errors and unused imports. Therefore, we apply a multi-label annotation in this study.

• **Taxonomy of Failure Patterns.** Based on the annotation results, Table 2 summarizes eight categories of failure patterns (P1-P8), and we introduce them as follows:

*P1: Syntax Error.* This category of pattern occurs when a generated code snippet is not executable due to syntax errors or incomplete snippets (e.g., mismatched indentation and truncated statements). In §3.2(b), an extra ‘]’ appears on Line 34, causing the code snippet to fail syntax checking. Such errors force developers to engage in iterations of locating, fixing, and rewriting, which not only prolongs debugging and integration processes but can also increase overall delivery costs and schedule risks. In addition, frequent naive errors may force developers to abandon the use of specific models or libraries, potentially affecting the digital market and software ecology.

*P2: Incorrect Library Usage.* This failure pattern occurs when a snippet contains the incorrect usage of a target library or its APIs, for example, invoking deprecated or non-existent functions, selecting APIs ill-suited to the input characteristics, or failing to call the target library as required. Compared with syntax errors, such a misuse is more insidious. It may lead to logical errors that are difficult to identify, posing security and reliability risks to software development.

*P3: Incorrect Functionality.* In this pattern, the code snippet generated by the LLM may be executable, but it implements incorrect algorithms and fails to meet the core task requirements. For instance, when asked to build a convolutional neural network in PyTorch, the model (e.g., Qwen3-Coder-Plus) may return a classifier with only fully connected layers and no convolutional layers, which is undoubtedly not enough for the task. This pattern is the most common across six LLMs, reflecting the limitations of LLMs in performing complex tasks relevant to production scenarios to a certain extent. Frequent functionality issues could force the developers to repeatedly rework and verify, erode trust in automated coding assistance, and increase testing and review overhead across the pipeline.

*P4: Missing Edge Handling.* The code snippet omits checks or handlers for edge cases and inputs (e.g., empty inputs and extreme values), making it less reliable and likely to fail under certain conditions. Such failures are often expensive to troubleshoot and difficult to expose in tests, posing a threat to the security and reliability of software development.

*P5: Inefficient Implementation.* This pattern is related to the performance efficiency dimension. It occurs when the code snippet contains inefficient implementations such as redundant and frequent I/O, nested loops, and exception checking. This directly leads to higher latency and resource



Table 2. Taxonomy of Failure Patterns

| No. | Failure Pattern            | Description                                                                                                  |
|-----|----------------------------|--------------------------------------------------------------------------------------------------------------|
| P1  | Syntax Error               | Containing syntax errors and missing content, making the code snippet inexecutable.                          |
| P2  | Incorrect Library Usage    | Containing incorrect usage of the libraries and corresponding APIs.                                          |
| P3  | Incorrect Functionality    | Containing incorrect implementation, resulting in a failure to meet the core requirements of the given task. |
| P4  | Missing Edge Handling      | Failing to consider or process the edge cases, which may still cause errors on specific inputs.              |
| P5  | Inefficient Implementation | Containing inefficient implementations, which can still be optimized.                                        |
| P6  | Missing Comments           | Lacking necessary comments and explanations for complex functions, classes, or key steps.                    |
| P7  | Import Issues              | Using libraries that are not imported, or the imported library is not used.                                  |
| P8  | Others                     | Other issues, such as containing too many demo inputs, leading to a low maintainability index.               |

consumption, which degrades user experience and software scalability, accrues as performance debt, making later optimization riskier and more expensive.

*P6: Missing Comments.* The code snippet provides insufficient documentation for complex functions, classes, or key steps. Missing or minimal comments will increase cognitive load and misuse risk and slow code review, harming the long-term usability of the code snippet.

*P7: Import Issues.* This failure pattern includes the situations where code snippets use APIs from specific libraries but do not import them, or import modules that are never used. These issues can cause execution failures, pollute the environment, and cause dependency bloat, complicating software deployment and leading to potential issues in subsequent version control and dependency.

*P8: Others.* This category includes various code quality problems, e.g., excessive demo inputs, which depress the scores in dimensions like maintainability. Such issues can hinder human developers from understanding the code snippets generated by LLMs, slow down their learning speed, thereby reducing project maintainability and increasing development costs.

• **Analysis of Failure Patterns.** Table 3 shows the distribution of failure cases across six LLMs. Our analysis shows that ‘Incorrect Functionality’ and ‘Missing Edge Handling’ are the two most common failure patterns, separately accounting for 53.07% and 40.66% of all failure cases. This finding is alarming, as these two types of issues are closely related to the quality and output of software development. Functional correctness is the cornerstone of software, while comprehensive edge case and exception handling are key to ensuring stable and reliable system operation in real-world industrial environments [23, 24]. When models systematically ignore these edge handling processes, they effectively plant a ‘ticking time bomb’ in the code snippets. It could lead to various hidden dangers during application deployment, ranging from security vulnerabilities caused by insufficient input validation to service crashes caused by uncaught exceptions. Therefore, one of the most serious dangers exposed by the low AI proficiency of a library is that it induces models to generate code snippets with inherent defects in functional correctness and reliability, thereby amplifying these security risks in the LLM era.

In addition, the pattern of ‘Syntax Errors’ accounts for a significant proportion of GPT-4o’s outputs. Among the 4,250 code snippets generated by GPT-4o, 250 of them contain syntax errors, accounting for 5.88%. These seemingly naive errors can directly lead to build or integration failures in real-world engineering processes, significantly extending debugging cycles and severely undermining development teams’ trust in AI-assisted development. For model providers, persistent syntax issues not

Table 3. Distribution of Failure Cases

| Model            | P1  | P2  | P3  | P4  | P5  | P6  | P7  | P8 | Total |
|------------------|-----|-----|-----|-----|-----|-----|-----|----|-------|
| GPT-4o-mini      | 15  | 31  | 89  | 81  | 73  | 50  | 34  | 2  | 375   |
| GPT-4o           | 177 | 42  | 154 | 58  | 37  | 15  | 25  | 0  | 508   |
| Claude Sonnet 4  | 51  | 19  | 94  | 77  | 62  | 62  | 31  | 12 | 408   |
| Gemini 2.5 Flash | 10  | 20  | 32  | 41  | 29  | 24  | 7   | 16 | 179   |
| Qwen3-Coder-Plus | 1   | 5   | 28  | 29  | 29  | 4   | 10  | 2  | 108   |
| DeepSeek-R1      | 25  | 11  | 35  | 45  | 36  | 28  | 13  | 5  | 198   |
| Total            | 279 | 128 | 432 | 331 | 266 | 183 | 120 | 37 | -     |

only mask deeper product flaws (e.g., low efficiency in the generated code snippets) but also undermine the model's iteration efficiency, user experience, and market reputation.

**Finding 4:** The most common failure patterns of low-quality code snippets are 'Incorrect Functionality' and 'Missing Edge Handling'. These can seriously endanger the correctness and reliability of software systems and introduce potential security issues to software development. It highlights the gap between LLM code generation results and the requirements of actual software development scenarios.

#### 5.4 RQ3: Prompting Techniques

To explore the impact of prompting techniques on AI proficiency of libraries, especially their effect on code quality gaps between competing libraries, we implement and evaluate three general prompting methods in this section. ❶ **CoT** is the zero-shot variant of Chain-of-Thought prompting, which includes the phrase 'Let's think step by step' in the system prompt [44]. It can encourage the model to think gradually and generate structured and detailed answers. Prior work has demonstrated that CoT can improve performance on reasoning and generation tasks [44, 84]. ❷ **Few-shot prompting** provides examples in the prompt and guides the model to generate high-quality answers. Following the prior work [47, 67, 87], for each prompt with a target library and a specific task, we provide a dedicated example containing a different task and a corresponding code snippet using the target library. This example code snippet achieves an overall quality score of over 80. ❸ **Regeneration** simulates the user workflow of 'regenerating when dissatisfied'. For each prompt, the model independently produces three candidate responses. We then select the one with the highest overall quality score as the final output.

We evaluate the above three methods on six LLMs. Due to resource constraints, we randomly select 20 pairs of prompts for each model. Each pair contains two competing libraries under the same task. 10 pairs exhibit a significant quality gap between their original generated code snippets without the prompting methods, whereas the other 10 pairs do not (Cohen's effect size). Following the setup in §5.1, we repeatedly query each prompt five times on each model. Table 4 shows the experimental results across six LLMs, where 'Score' represents the average overall code quality scores across LLMs, 'Gap' indicate the absolute difference in overall code quality scores between competing library pairs, and 'Cons.' represents the proportion of pairs whose leading relationship of code quality scores between two libraries remains unchanged after using different prompting methods. 'Origin' represents the results without using any prompting method. 'Regen.' is the abbreviation of the regeneration method.

• **Analysis of Code Quality Score.** As shown in Table 4, prompt techniques can significantly improve the quality of code snippets generated by LLMs. Specifically, 'Few-shot' and 'Regeneration' significantly improve the overall quality score (Wilcoxon signed-rank test,  $p < 0.05$ ), with the largest improvement reaching 5.38% ('Few-shot' on GPT-4o). In addition, both methods significantly improve the scores on other quality dimensions such as functional suitability and Reliability. In contrast, 'CoT' only provides slight gains on several models, and the improvement is small (the improved score is less than 1). This may be because the advantage of 'CoT' primarily lies in 'explicit reasoning', while this study focuses more on quality dimensions such as the correctness and maintainability of implementation. In such a situation, additional thought chains and reasoning may not improve the code quality but could lead to more complex implementations, increasing complexity and compromising maintainability scores, ultimately resulting in little improvement. Developers in the LLM era should choose appropriate prompting techniques based on the actual requirements of the coding task to reach high AI proficiency and improve development productivity.

Table 4. Impact of Prompting Techniques on Competing Libraries

| Method   | GPT-4o-mini |      |           | GPT-4o |      |           | Claude Sonnet 4 |      |           | Gemini 2.5 Flash |      |           | Qwen3-Coder-Plus |      |           | DeepSeek-R1 |      |           |
|----------|-------------|------|-----------|--------|------|-----------|-----------------|------|-----------|------------------|------|-----------|------------------|------|-----------|-------------|------|-----------|
|          | Score       | Gap  | Cons. (%) | Score  | Gap  | Cons. (%) | Score           | Gap  | Cons. (%) | Score            | Gap  | Cons. (%) | Score            | Gap  | Cons. (%) | Score       | Gap  | Cons. (%) |
| Origin   | 74.08       | 6.23 | -         | 77.01  | 8.47 | -         | 78.07           | 4.72 | -         | 81.06            | 4.10 | -         | 77.42            | 5.96 | -         | 78.87       | 5.68 | -         |
| CoT      | 73.44       | 3.77 | 65.00     | 77.87  | 5.48 | 55.00     | 77.59           | 2.78 | 60.00     | 80.85            | 2.43 | 55.00     | 78.00            | 4.05 | 75.00     | 79.07       | 3.77 | 45.00     |
| Few-shot | 75.90       | 5.04 | 50.00     | 81.15  | 4.97 | 45.00     | 80.04           | 2.82 | 30.00     | 82.41            | 3.33 | 50.00     | 80.35            | 3.28 | 70.00     | 81.15       | 4.27 | 60.00     |
| Regen.   | 77.08       | 6.00 | 75.00     | 80.82  | 6.30 | 85.00     | 79.76           | 4.69 | 95.00     | 82.68            | 4.50 | 85.00     | 79.86            | 5.45 | 85.00     | 80.59       | 5.31 | 100.00    |

**Finding 5:** ‘Few-shot’ and ‘Regeneration’ significantly improve overall code quality on multiple LLMs, leading to better AI proficiency, while ‘CoT’ is less effective in improving code quality.

• **Analysis of Competing Libraries.** We calculate the average code quality differences (absolute gaps) for competing library pairs, as shown in Table 4. We can observe that both ‘CoT’ and ‘Few-shot’ methods significantly reduce the quality gap between libraries ( $p < 0.05$ ). The ‘Few-shot’ method, in particular, reduces the average gap by up to 44.93% on Qwen3-Coder-Plus. This provides a viable path to mitigate the potential ‘winner-takes-all’ problem (Matthew effect) caused by differences in AI proficiency. It also suggests that in the LLM era, when selecting a technology stack, prompting techniques can be used as an auxiliary means to regulate the AI proficiency of target libraries. Note that the impact of prompting techniques is conditional. For competing libraries with significant code quality gaps without prompting, all three prompting methods are effective in narrowing the gap. However, for pairs with a narrow gap, applying prompting methods can actually lead to a ‘lead reversal’ (where the score of library A is better initially, but library B achieves a higher score after prompting) and even widen the gap. This phenomenon is most pronounced for the ‘Few-shot’ method. The reason may be that the performance of few-shot prompts is highly dependent on the content and coverage of the selected examples. When examples better align with a library’s API conventions or usage patterns, they can greatly improve the quality of code generated by LLMs using that library, thereby changing the relative advantages of two libraries.

**Finding 6:** Prompt techniques such as ‘Few-shot’ and ‘CoT’ can narrow the AI proficiency gap between competing libraries. However, for pairs with narrow gaps, prompting methods may lead to a shift in the leading relationship between the scores of libraries, or even widen the gap.

6 Discussion

• **AI Proficiency and Popularity.** Intuitively, the AI proficiency of one library should be related to its popularity. More popular libraries have higher visibility in open-source communities, leading to more examples and documentation. Therefore, they are more likely to be included in training corpora, enabling LLMs to more fully learn and effectively use the library. To examine the correlation between AI proficiency and popularity, we collect the number of GitHub stars as the popularity signal for each library. For each pair of competing libraries from §5.2, we use exact two-sided binomial tests to assess whether AI proficiency rankings align with popularity rankings. The concordance rate is 0.574 (95% confidence interval [0.503, 0.643]), with significant results ( $p < 0.05$ ) but very small effect size (Cohen’s  $d = 0.15$ ), indicating a statistically significant yet weak positive correlation between library popularity and AI proficiency. Such observation further supports the viewpoint that popular libraries good for human users may not be suitable for LLM-driven workflows, further emphasizing the importance of incorporating AI proficiency into technology selection frameworks. Furthermore, we observe that among 80 competing library pairs, the library with more GitHub stars actually receives lower AI proficiency scores. For example, the data management and tracking library DVC has 2.12 times more GitHub stars than Tensorboard, but its AI

proficiency score on GPT-4o is 6.46% lower than that of Tensorboard. This divergence may stem from a variety of factors, including differences in the quality of use cases across different libraries in the open-source community, data filtering strategies used during LLM pre-training, and API iterations caused by rapid software updates. In summary, technology selection solely from the developer's perspective is insufficient in the context of AI-assisted development. Incorporating AI proficiency as a new evaluation dimension helps teams assess whether LLMs can effectively use a given technology, avoiding debugging overhead and technical debt.

• **Implications.** In the LLM era, software development processes are undergoing a transformation, shifting from a traditional process dominated solely by developers to a collaboration process between developers and LLMs. Our study has three practical implications.

❶ *For Developers: Rethinking Technology Selection.* Our findings show that technologies that are easy for human developers to use may not be equally effective for LLMs and fail to produce high-quality code snippets. This discrepancy reveals the limitations of existing technology selection methods, which focus solely on the intrinsic attributes of technologies (e.g., performance), ignoring whether LLMs can effectively leverage these technologies in code generation. As the AI-assisted development becomes increasingly common, ignoring this discrepancy can lead to increased debugging costs and even the accumulation of unforeseen technical debt. We urge developers to incorporate the new dimension of 'AI coding proficiency' into technology selection, thereby selecting appropriate technologies in the LLM era.

❷ *For Software Community: Beware of Ecosystem Imbalance.* As LLMs are commonly used in development, the AI-proficient libraries may rapidly gain more users and resources, creating a 'winner-takes-all' situation. This not only inhibits technological diversity but also potentially leads to legal consequences. We recommend that software communities and maintainers acknowledge the ecological risks posed by disparities in AI proficiency and design mitigation mechanisms, for example, publishing high-quality few-shot examples for different libraries to help users achieve better LLM code generation results and narrow the AI proficiency gap with leading libraries.

❸ *For Software Security: Beware of Emerging Risk Entrances.* While AI-assisted development improves efficiency, it also introduces new security risks. On the one hand, the code generated by LLMs may lack critical logic (e.g., exception handling), allowing undetected vulnerabilities to infiltrate the system. On the other hand, excessive reliance on a small number of libraries with high proficiency can amplify the potential harm of supply chain attacks (e.g., dependency injection or poisoning) [25, 63]. Therefore, we call on incorporating AI coding proficiency into the governance and compliance assessment of AI systems, and the audit and control of LLM-generated content should be strengthened to ensure software security and supply chain resilience.

• **Future Directions.** Future research can be carried out in the following directions:

❶ *Comprehensive benchmark for AI proficiency.* Although this paper defines AI proficiency and reveals the proficiency gaps between competing libraries on six popular LLMs, our study is limited to the Python programming language and third-party libraries. Future research could develop a larger-scale, cross-platform, and cross-language (e.g., C++) benchmark encompassing more technology types, including platforms and systems. It should also incorporate complex tasks requiring the collaboration of multiple technologies to better approximate real production environments. Furthermore, the benchmark could incorporate additional quality dimensions (e.g., cross-platform portability) to better assist developers in AI-assisted technology selection.

❷ *New technology selection framework.* In addition to AI proficiency, the AI-assisted development raises other new dimensions of technology stack selection that need to be explored, for example, AI efficiency (how long and how many tokens it takes for the model to generate code snippets that meet requirements) and agent proficiency (whether a tool is ready for efficient invocation from the agentic AI). Future research could extend existing methodologies (e.g., ATAM) to develop a comprehensive

technology stack selection framework tailored to the LLM era. This framework would integrate new dimensions, including AI proficiency, with traditional attributes (e.g., performance), offering a multidimensional approach with practical value for efficient software development.

③ *Enhancement strategy.* Another valuable future direction is to design methods to improve the AI proficiency of target libraries on a given model. There are two potential approaches. On the model side, model developers can augment training data and construct comparative fine-tuning to mitigate the AI proficiency differences between competing libraries. On the library side, library maintainers can provide high-quality use cases and documentation to the open-source community and improve the interfaces and corresponding annotations to make the library-related data easier to learn and use for LLMs. The coordinated implementation of these two approaches has the potential to narrow the AI proficiency gap between libraries, mitigate risks posed to the software ecosystem, and improve developer experience and delivery quality.

## 7 Conclusion

In this paper, we propose a new perspective for technology stack selection in the LLM era, *AI coding proficiency*, which evaluates whether LLMs can correctly utilize the given technology in code generation, thereby producing high-quality code snippets. We conduct the first empirical study on the AI proficiency of third-party libraries across six LLMs. Our findings reveal that competing libraries with similar functionality can exhibit significant gaps in AI proficiency on the same model, and the performance of different models using the same library can also vary. Low AI proficiency not only undermines the efficiency and effectiveness of AI-assisted development, resulting in additional time and financial costs, but can also drive users toward high-proficiency libraries and exacerbate market concentration, ultimately harming technological diversity and security of the software ecosystem. We call on the community to take heed of and incorporate AI proficiency into the technology selection framework and take practical improvement measures to ensure development efficiency and delivery quality, and maintain an open and diverse software ecosystem.

## References

- [1] 1914. Federal Trade Commission Act.
- [2] 2024. What is AI readiness? <https://www.ataccama.com/blog/ai-readiness>.
- [3] 2025. Our repository. <https://anonymous.4open.science/r/AICodingProficiency-15C0/>.
- [4] Aider. 2025. Aider LLM Leaderboards. <https://aider.chat/docs/leaderboards/>
- [5] Anthropic. 2025. Claude 4 Sonnet. <https://www.anthropic.com/claude/sonnet>
- [6] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732* (2021).
- [7] Cornel Barna, Marin Litoiu, and Hamoun Ghanbari. 2011. Autonomic load-testing framework. In *Proceedings of the 8th ACM international conference on Autonomic computing*. 91–100.
- [8] Len Bass, Paul Clements, and Rick Kazman. 2021. *Software architecture in practice*. Addison-Wesley Professional.
- [9] Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. 2023. Prompting is programming: A query language for large language models. *Proceedings of the ACM on Programming Languages* 7, PLDI (2023), 1946–1969.
- [10] Nick D Birrell, Martyn A Ould, and MA Ould. 1988. *A practical handbook for software development*. Cambridge University Press.
- [11] Lionel C. Briand, John W. Daly, and Jurgen K Wust. 2002. A unified framework for coupling measurement in object-oriented systems. *IEEE Transactions on software Engineering* 25, 1 (2002), 91–121.
- [12] Lei Chai and Ming Li. 2022. Pyramid attention for source code summarization. *Advances in Neural Information Processing Systems* 35 (2022), 20421–20433.
- [13] Pierre Chambon, Baptiste Roziere, Benoit Sagot, and Gabriel Synnaeve. 2025. BigO (Bench)—Can LLMs Generate Code with Controlled Time and Space Complexity? *arXiv preprint arXiv:2503.15242* (2025).
- [14] Liguang Chen, Qi Guo, Hongrui Jia, Zhengran Zeng, Xin Wang, Yijiang Xu, Jian Wu, Yidong Wang, Qing Gao, Jindong Wang, et al. 2024. A survey on evaluating large language models in code generation tasks. *arXiv preprint arXiv:2408.16498* (2024).
- [15] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [16] Alibaba Cloud. 2025. Qwen3-Coder-Plus. [https://modelstudio.console.alibabacloud.com/?spm=a3c0i.29328889.9901980110.3.3f962b6akYcpTR&tab=doc#/doc/?type=model&url=2840914\\_2&modelId=qwen3-coder-plus](https://modelstudio.console.alibabacloud.com/?spm=a3c0i.29328889.9901980110.3.3f962b6akYcpTR&tab=doc#/doc/?type=model&url=2840914_2&modelId=qwen3-coder-plus)
- [17] Jacob Cohen. 2013. *Statistical power analysis for the behavioral sciences*. routledge.
- [18] Tristan Coignion, Clément Quinton, and Romain Rouvoy. 2024. A performance study of llm-generated code on leetcode. In *Proceedings of the 28th international conference on evaluation and assessment in software engineering*. 79–89.
- [19] Thomas H Cormen, Charles E Leiserson, Ronald L Rivest, and Clifford Stein. 2022. *Introduction to algorithms*. MIT press.
- [20] Domenico Cotroneo, Luigi De Simone, Pietro Liguori, and Roberto Natella. 2020. Fault injection analytics: A novel approach to discover failure modes in cloud-computing systems. *IEEE transactions on dependable and secure computing* 19, 3 (2020), 1476–1491.
- [21] Robert W Crandall and Thomas W Hazlett. 2023. Antitrust reform in the digital era: A skeptical perspective. *The University of Chicago Business Law Review* 2, 2 (2023), 1.
- [22] Giuseppe Crupi, Rosalia Tufano, Alejandro Velasco, Antonio Mastropaolo, Denys Poshyvanyk, and Gabriele Bavota. 2025. On the Effectiveness of LLM-as-a-judge for Code Generation and Summarization. *IEEE Transactions on Software Engineering* (2025).
- [23] Guilherme B De Padua and Weiyi Shang. 2017. Revisiting exception handling practices with exception flow analysis. In *2017 IEEE 17th International working conference on source code analysis and manipulation (SCAM)*. IEEE, 11–20.
- [24] Guilherme B de Padua and Weiyi Shang. 2018. Studying the relationship between exception handling practices and post-release defects. In *Proceedings of the 15th International Conference on Mining Software Repositories*. 564–575.
- [25] Alexandre Decan, Tom Mens, and Eleni Constantinou. 2018. On the impact of security vulnerabilities in the npm package dependency network. In *Proceedings of the 15th international conference on mining software repositories*. 181–191.
- [26] Yashar Deldjoo. 2024. Understanding biases in chatgpt-based recommender systems: Provider fairness, temporal stability, and recency. *ACM Transactions on Recommender Systems* (2024).
- [27] Dirk Draheim, John Grundy, John Hosking, Christof Lutteroth, and Gerald Weber. 2006. Realistic load testing of web applications. In *Conference on Software Maintenance and Reengineering (CSMR'06)*. IEEE, 11–pp.
- [28] Thomas D Gauthier. 2001. Detecting trends using Spearman’s rank correlation coefficient. *Environmental forensics* 2, 4 (2001), 359–362.



- [29] Bruce L Golden, Edward A Wasil, and Patrick T Harker. 1989. The analytic hierarchy process. *Applications and Studies, Berlin, Heidelberg* 2, 1 (1989), 1–273.
- [30] Google. 2025. Gemini 2.5 Flash. <https://cloud.google.com/vertex-ai/generative-ai/docs/models/gemini/2-5-flash>
- [31] Tovi Grossman, George Fitzmaurice, and Ramtin Attar. 2009. A survey of software learnability: metrics, methodologies and guidelines. In *Proceedings of the sigchi conference on human factors in computing systems*. 649–658.
- [32] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948* (2025).
- [33] Adam Hales. 2025. "Unstoppable?" ChatGPT Surges to 700 million Weekly Users as Rivals Race to Compete. <https://www.windowscentral.com/artificial-intelligence/chatgpt-is-set-to-hit-700-million-weekly-users-but-can-its-rivals-catch-up>
- [34] Maurice H Halstead. 1977. *Elements of Software Science (Operating and programming systems series)*. Elsevier Science Inc.
- [35] Ahmed E Hassan. 2009. Predicting faults using the complexity of code changes. In *2009 IEEE 31st international conference on software engineering*. IEEE, 78–88.
- [36] Wenpin Hou and Zhicheng Ji. 2025. Comparing large language models and human programmers for generating programming code. *Advanced Science* 12, 8 (2025), 2412279.
- [37] Dong Huang, Yuhao Qing, Weiye Shang, Heming Cui, and Jie M Zhang. 2024. Effibench: Benchmarking the efficiency of automatically generated code. *Advances in Neural Information Processing Systems* 37 (2024), 11506–11544.
- [38] International Organization for Standardization. 2011. ISO/IEC 25010:2011 Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - System and software quality models. [Online]. Available: <https://www.iso.org/standard/35733.html>.
- [39] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2024. A survey on large language models for code generation. *arXiv preprint arXiv:2406.00515* (2024).
- [40] Siyuan Jin, Ziyuan Li, Bichao Chen, Bing Zhu, and Yong Xia. 2023. Software code quality measurement: Implications from metric distributions. In *2023 IEEE 23rd International Conference on Software Quality, Reliability, and Security (QRS)*. IEEE, 488–496.
- [41] Jan Jöhnk, Malte Weißert, and Katrin Wyrski. 2021. Ready or not, AI comes: an interview study of organizational AI readiness factors. *Business & information systems engineering* 63, 1 (2021), 5–20.
- [42] Rick Kazman, Jai Asundi, and Mark Klein. 2001. Quantifying the costs and benefits of architectural decisions. In *Proceedings of the 23rd International Conference on Software Engineering. ICSE 2001*. IEEE, 297–306.
- [43] Rick Kazman, Mark Klein, and Paul Clements. 2000. *ATAM: Method for architecture evaluation*. Technical Report.
- [44] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large language models are zero-shot reasoners. *Advances in neural information processing systems* 35 (2022), 22199–22213.
- [45] Jean-Louis Letouzey and Michel Ilkiewicz. 2012. Managing technical debt with the sqale method. *IEEE software* 29, 6 (2012), 44–51.
- [46] Bowen Li, Wenhan Wu, Ziwei Tang, Lin Shi, John Yang, Jinyang Li, Shunyu Yao, Chen Qian, Binyuan Hui, Qicheng Zhang, et al. 2025. Prompting Large Language Models to Tackle the Full Software Development Lifecycle: A Case Study. In *Proceedings of the 31st International Conference on Computational Linguistics*. 7511–7531.
- [47] Jia Li, Chongyang Tao, Jia Li, Ge Li, Zhi Jin, Huangzhao Zhang, Zheng Fang, and Fang Liu. 2023. Large language model-aware in-context learning for code generation. *ACM Transactions on Software Engineering and Methodology* (2023).
- [48] R Li, LB Allal, Y Zi, N Muennighoff, D Kocetkov, C Mou, M Marone, C Akiki, J Li, J Chim, et al. 2023. StarCoder: May the Source be With You! *Transactions on machine learning research* (2023).
- [49] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. 2022. Competition-level code generation with alphacode. *Science* 378, 6624 (2022), 1092–1097.
- [50] Amjad Masad. 2025. Replit CEO says anyone can build an app in an afternoon with AI and it's powering a new wave of solo startups. Business Insider (online). <https://www.businessinsider.com/replit-ceo-vibe-coding-solo-startups-2025-8>
- [51] Thomas J McCabe. 1976. A complexity measure. *IEEE Transactions on software Engineering* 4 (1976), 308–320.
- [52] Catherine C McGeoch. 2012. *A guide to experimental algorithmics*. Cambridge University Press.
- [53] Robert K Merton. 1968. The Matthew effect in science: The reward and communication systems of science are considered. *Science* 159, 3810 (1968), 56–63.
- [54] Christopher Z Mooney, Robert D Duval, and Robert Duvall. 1993. *Bootstrapping: A nonparametric approach to statistical inference*. Number 95. sage.
- [55] Nachiappan Nagappan and Thomas Ball. 2005. Use of relative code churn measures to predict system defect density. In *Proceedings of the 27th international conference on Software engineering*. 284–292.

- [56] Roberto Natella, Domenico Cotroneo, and Henrique S Madeira. 2016. Assessing dependability with software fault injection: A survey. *ACM Computing Surveys (CSUR)* 48, 3 (2016), 1–55.
- [57] Meghan Neville. 2025. Why Most CTOs Pick the Wrong Technology Stack (And How to Avoid Their Mistakes). <https://softjournal.com/insights/choose-the-right-tech-stack>
- [58] Paul Oman and Jack Hagemester. 1992. Metrics for assessing a software system’s maintainability. In *Proceedings Conference on Software Maintenance 1992*. IEEE Computer Society, 337–338.
- [59] OpenAI. 2024. GPT-4o mini: advancing cost-efficient intelligence. <https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/>
- [60] OpenAI. 2024. GPT-4o System Card. <https://openai.com/index/gpt-4o-system-card/>
- [61] OpenAI. 2025. Introducing GPT-5 for developers. OpenAI blog. <https://openai.com/index/introducing-gpt-5-for-developers/>
- [62] Stack Overflow. 2025. Developer Survey. <https://survey.stackoverflow.co/2025/ai>
- [63] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2022. Asleep at the keyboard? assessing the security of github copilot’s code contributions. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 754–768.
- [64] Yun Peng, Jun Wan, Yichen Li, and Xiaoxue Ren. 2025. Coffe: A code efficiency benchmark for code generation. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 242–265.
- [65] Marco Piccioni, Carlo A Furia, and Bertrand Meyer. 2013. An empirical study of API usability. In *2013 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*. IEEE, 5–14.
- [66] Daryl Posnett, Abram Hindle, and Premkumar Devanbu. 2011. A simpler model of software readability. In *Proceedings of the 8th working conference on mining software repositories*. 73–82.
- [67] PromptHub. 2025. The Few Shot Prompting Guide. <https://www.prompthub.us/blog/the-few-shot-prompting-guide#few-shot-prompting-examples>
- [68] Yuhao Qing, Boyu Zhu, Mingzhe Du, Zhijiang Guo, Terry Yue Zhuo, Qianru Zhang, Jie M Zhang, Heming Cui, Siu-Ming Yiu, Dong Huang, et al. 2025. EffiBench-X: A Multi-Language Benchmark for Measuring Efficiency of LLM-Generated Code. *arXiv preprint arXiv:2505.13004* (2025).
- [69] Irum Rauf, Elena Troubitsyna, and Ivan Porres. 2019. A systematic mapping study of API usability evaluation methods. *Computer Science Review* 33 (2019), 49–68.
- [70] Francisco Ribeiro. 2023. Large language models for automated program repair. In *Companion Proceedings of the 2023 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity*. 7–9.
- [71] Ruksit Rojpaisarnkit, Gregorio Robles, Raula Gaikovina Kula, Dong Wang, Chaiyong Ragkhitwetsagul, Jesus M Gonzalez-Barahona, and Kenichi Matsumoto. 2024. Towards Identifying Code Proficiency Through the Analysis of Python Textbooks. In *2024 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 288–299.
- [72] Philip Sedgwick. 2014. Spearman’s rank correlation coefficient. *Bmj* 349 (2014).
- [73] Bles Software. 2025. Startup Tech Stack 2025: Choosing the Right Technologies for Your Product. <https://www.bles-software.com/blog/startup-tech-stack-2025-choosing-the-right-technologies-for-your-product>
- [74] Stratoflow. 2023. SDLC Guide: How to Conduct Software Design Phase. Technical guidance. <https://stratoflow.com/sdlc-design-phase/>
- [75] Kunsheng Tang, Wenbo Zhou, Jie Zhang, Aishan Liu, Gelei Deng, Shuai Li, Peigui Qi, Weiming Zhang, Tianwei Zhang, and Nenghai Yu. 2024. GenderCARE: A Comprehensive Framework for Assessing and Reducing Gender Bias in Large Language Models. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 1196–1210.
- [76] Ali N Tehrani, Subhasis Ray, Sanjit K Roy, Richard L Gruner, and Francesco P Appio. 2024. Decoding AI readiness: An in-depth analysis of key dimensions in multinational corporations. *Technovation* 131 (2024), 102948.
- [77] TESTINGMIND. 2025. ChatGPT for Automated Testing: From Conversation to Code. <https://www.testingmind.com/chatgpt-for-automated-testing-from-conversation-to-code/>
- [78] Weixi Tong and Tianyi Zhang. 2024. CodeJudge: Evaluating Code Generation with Large Language Models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 20032–20051.
- [79] John Wilder Tukey et al. 1977. *Exploratory data analysis*. Vol. 2. Springer.
- [80] Lukas Twist, Jie M Zhang, Mark Harman, Don Syme, Joost Noppen, and Detlef Nauck. 2025. LLMs Love Python: A Study of LLMs’ Bias for Programming Languages and Libraries. *arXiv preprint arXiv:2503.17181* (2025).
- [81] Gwo-Hshiung Tzeng and Jih-Jeng Huang. 2011. *Multiple attribute decision making: methods and applications*. CRC press.
- [82] vellum. 2025. LLM Leaderboard. <https://www.vellum.ai/llm-leaderboard>

- [83] Ruiqi Wang, Jiyu Guo, Cuiyun Gao, Guodong Fan, Chun Yong Chong, and Xin Xia. 2025. Can llms replace human evaluators? an empirical study of llm-as-a-judge in software engineering. *Proceedings of the ACM on Software Engineering* 2, ISSTA (2025), 1955–1977.
- [84] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
- [85] Colin White, Samuel Dooley, Manley Roberts, Arka Pal, Benjamin Feuer, Siddhartha Jain, Ravid Shwartz-Ziv, Neel Jain, Khalid Saifullah, Sreemanti Dey, et al. 2025. LiveBench: A Challenging, Contamination-Limited LLM Benchmark. In *The Thirteenth International Conference on Learning Representations*.
- [86] Xin Xia, Zhiyuan Wan, Pavneet Singh Kochhar, and David Lo. 2019. How practitioners perceive coding proficiency. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 924–935.
- [87] Derek Xu, Tong Xie, Botao Xia, Haoyu Li, Yunsheng Bai, Yizhou Sun, and Wei Wang. 2024. Does Few-Shot Learning Help LLM Performance in Code Synthesis? *arXiv preprint arXiv:2412.02906* (2024).
- [88] Xiangzhe Xu, Zhuo Zhang, Zian Su, Ziyang Huang, Shiwei Feng, Yapeng Ye, Nan Jiang, Danning Xie, Siyuan Cheng, Lin Tan, et al. 2025. Unleashing the Power of Generative Model in Recovering Variable Names from Stripped Binary. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*.
- [89] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).
- [90] Alexander Zhang, Marcus Dong, Jiaheng Liu, Wei Zhang, Yejie Wang, Jian Yang, Ge Zhang, Tianyu Liu, Zhongyuan Peng, Yingshui Tan, et al. 2025. Codecriticbench: A holistic code critique benchmark for large language models. *arXiv preprint arXiv:2502.16614* (2025).
- [91] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. [n. d.]. RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation. In *The 2023 Conference on Empirical Methods in Natural Language Processing*.
- [92] Xiaoyu Zhang, Juan Zhai, Shiqing Ma, Qingshuang Bao, Weipeng Jiang, Qian Wang, Chao Shen, and Yang Liu. 2025. The Invisible Hand: Unveiling Provider Bias in Large Language Models for Code Generation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Vienna, Austria, 21376–21403. doi: 10.18653/v1/2025.acl-long.1038